4. Manual del simulador Flutter

4.1. Descripción

El **simulador** es un **servidor ADS** escrito en Dart/Flutter que emula un PLC

TwinCAT. No es un cliente: escucha conexiones TCP y responde a las peticiones ADS

de `LibPhithon` (Python) y `LibC#` (C#).

4.2. Requisitos

- SDK de Flutter (incluye Dart). El modo headless solo necesita Dart.
- Para la interfaz gráfica: un destino de escritorio (`windows`, `linux` o `macos`).

4.3. Ejecución

Modo headless (sin interfaz, solo Dart)

```
cd simulador_flutter
dart bin/server.dart            # puerto 48898
dart bin/server.dart 48000      # puerto personalizado
```

Modo con interfaz (Flutter)

```
cd simulador_flutter
flutter run -d windows
```

4.4. Configuración de red

Parámetro	Valor
Dirección TCP	`127.0.0.1:48898` (configurable)
AmsNetId	`127.0.0.1.1.1`
Puerto AMS (runtime PLC)	`851`

Los clientes deben conectar su socket TCP a `48898` y dirigir sus cabeceras AMS al

NetId `127.0.0.1.1.1` y puerto AMS `851`.

4.5. Variables simuladas

Símbolo	Tipo	Valor inicial	Comportamiento
`MAIN.bStart`	BOOL	`false`	Escritura libre
`MAIN.bStop`	BOOL	`true`	Escritura libre
`MAIN.nCounter`	UINT	`0`	Se incrementa cada

			500 ms en estado RUN
`MAIN.iLevel`	INT	`100`	Escritura libre
`MAIN.fTemp`	REAL	`21.5`	Escritura libre
`MAIN.dPosition`	LREAL	`1.2345`	Escritura libre
`MAIN.sMessage`	STRING(80)	`"Hola ADS"`	Escritura libre
`MAIN.arrValues`	ARRAY[0..3] OF INT	`[1, 2, 3, 4]`	Escritura libre

4.6. Comandos ADS implementados

Comando	**Descripción**
`READ_DEVICE_INFO` (0x0001)	Devuelve versión 3.1 y nombre `ADS-Simulator`.
`READ_STATE` (0x0004)	Devuelve el estado ADS y de dispositivo.
`WRITE_CONTROL` (0x0005)	Cambia el estado (RUN/STOP/RESET…).
`READ/WRITE` (0x0002/0x0003)	Por nombre (0xF005), por handle (0xF004), handle-por-nombre (0xF003), info de símbolo (0xF006), tabla de símbolos (0xF000), imagen de proceso (0xF020/0xF030), y release de handle (0xF014).
`READ_WRITE` (0x0009)	Lee y escribe en una transacción.
`SUMUP_READ` (0xF080)	Lectura múltiple.
`SUMUP_WRITE` (0xF081)	Escritura múltiple.
`ADD/DEL/DEVICE_NOTIFICATION`	Notificaciones por cambio y por ciclo.

4.7. Notificaciones

- Para suscribirse a una variable por nombre, el cliente primero consulta

`SYM_INFOBYNAME` (obtiene `index_group=0xF005` e `index_offset=<índice>`) y luego

llama a `ADD_DEVICE_NOTIFICATION` con ese par índice/offset.

- El modo **on change** (por defecto) entrega un stamp cuando el valor cambia.
- El modo **cycle** entrega stamps periódicamente según `cycle_time`.

4.8. Estructura de archivos

Archivo	**Qué hace**
`bin/server.dart`	Servidor headless (Dart puro).
`lib/main.dart`	Interfaz gráfica Flutter.
`lib/ads/ams_header.dart`	Encode/decode de frame AMS/TCP y cabecera AMS.
`lib/ads/constants.dart`	Comandos, grupos de índice, estados,

	puertos y errores.
`lib/ads/datatypes.dart`	Codificación de tipos PLC.
`lib/ads/ads_client.dart`	Cliente ADS para el modo "PLC Real".
`lib/simulator/symbol_table.dart`	Tabla de símbolos y handles.
`lib/simulator/device.dart`	Estado del PLC, variables y notificaciones.
`lib/simulator/ads_server.dart`	Servidor TCP y despacho de comandos.

4.9. Interfaz gráfica

La UI permite:

- **Iniciar/Detener** y elegir **Modo** (PLC Simulado / PLC Real).
- Configurar **NetId AMS, Puerto AMS, IP y Puerto TCP**.
- **Buscar** variables y **seleccionar** las que quieras usar (checkbox).
- Elegir el **método** de cada variable seleccionada: **Leer**, **Escribir** o **Ambos**.
- **Expandir estructuras** (flecha ▶) para ver/editar cada miembro por separado.
- **Lanzar el cliente** Python/C# elegido.
- Consultar el **registro** de conexiones y tramas.

4.10. Modo PLC Simulado vs PLC Real

Modo	Comportamiento
PLC Simulado	El simulador actúa como **servidor** ADS; los clientes se conectan a él.
PLC Real	El simulador actúa como **cliente** ADS: se conecta a tu PLC TwinCAT real y lee/escribe sus variables.

En **PLC Real**, la tabla se actualiza sola cada segundo leyendo **solo las variables seleccionadas** (método Leer/Ambos). Al escribir una variable seleccionada (método Escribir/Ambos), la escritura se envía al PLC real.

4.11. Selección de variables y estructuras

- Tras **Cargar** un `variables.json` (generado desde el `.tpy`), se listan los símbolos de nivel superior (y los miembros expandidos).

- Marca con el **checkbox** las variables que quieras usar; no se leen todas.
- Cada variable seleccionada tiene un **método**: `Leer`, `Escribir` o `Ambos`.
- Las **estructuras** (tipos propios) se muestran en azul con una flecha ▶; al

pulsarla se expanden sus miembros (nombres con punto, p. ej.

`GVL.Estructura.Campo`), cada uno seleccionable y editable por separado.

4.12. Convertir un .tpy de TwinCAT

```
cd simulador_flutter
python convertir_tpy.py PLC_Portale.tpy variables.json
```

Genera un `variables.json` con dos secciones:

- `types`: definición de las estructuras (miembros con nombre, tipo y tamaño).
- `variables`: símbolos de nivel superior (nombre, tipo y tamaño).